

1. (a)

```

1 p1 = m.allocate(6);
2 p2 = m.allocate(3);
3 p3 = m.allocate(2);
4 m.deallocate(p2);
5 p4 = m.allocate(4);
6 m.deallocate(p1);

```

(Without Coalescing)

0.

Initial	
freeStart: 0	
0	15 size(heap)
1	-1 point to next free list
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

1.

(Success)

p1 = m.allocate(6)	
freeStart: 7	
0	7 6+1
1	
2	
3	
4	
5	
6	
7	8
8	-1
9	
10	
11	
12	
13	
14	

2.

(Success)

p2 = m.allocate(3)	
freeStart: 11	
0	7
1	
2	
3	
4	
5	
6	
7	4
8	
9	
10	
11	4
12	-1
13	
14	

1. (10 points) Given the sequence of memory allocation and deallocation requests as follows. Let the heap size be 15 and the heap grows downward (from index 0).

```

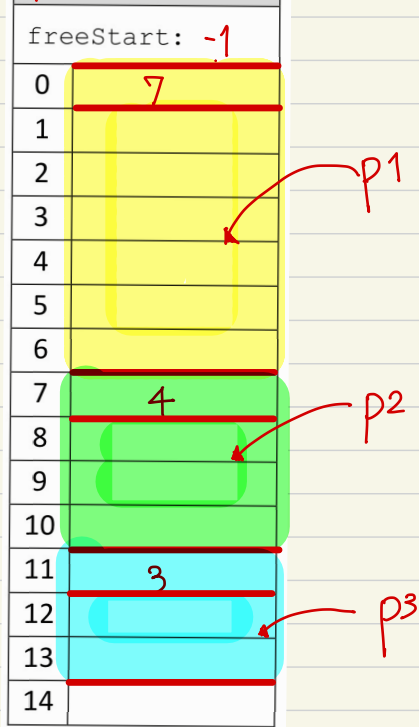
1 p1 = m.allocate(6);
2 p2 = m.allocate(3);
3 p3 = m.allocate(2);
4 m.deallocate(p2);
5 p4 = m.allocate(4);
6 m.deallocate(p1);

```

- a. (5 points) Provide a step-by-step demonstration of the allocation/deallocation sequence using the *first-fit algorithm* **without** coalescing. Indicate whether such an allocation/deallocation is successful.

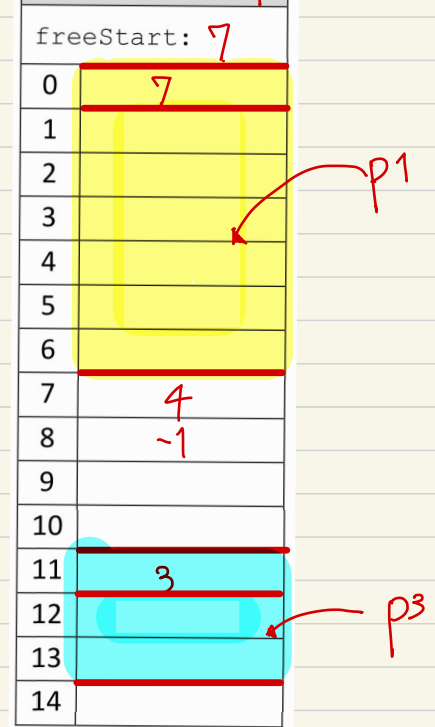
(Success)

3. $p3 = m.allocate(20)$



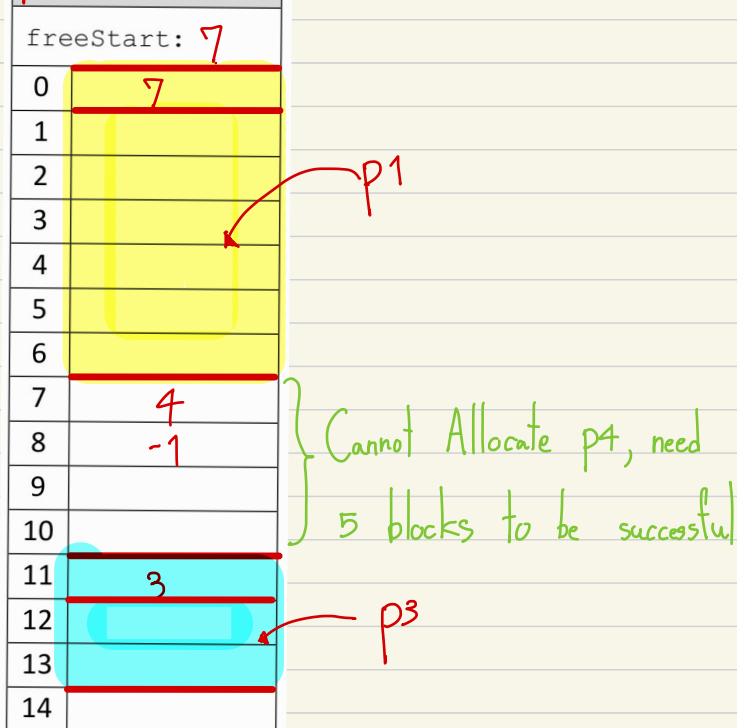
(Success)

4. $m.deallocate(p2)$



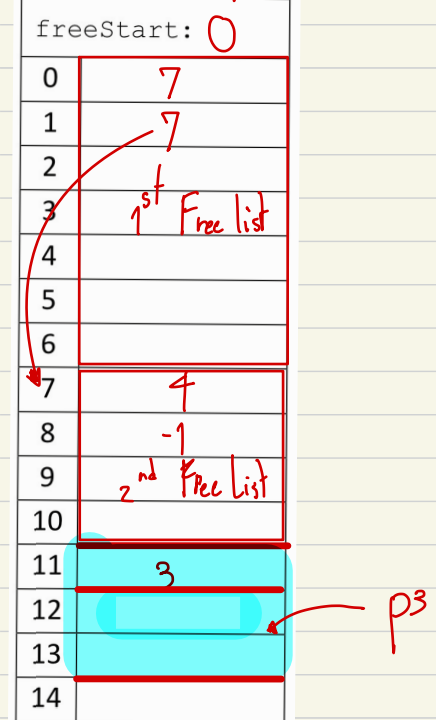
(Fail)

5. $p4 = m.allocate(4)$



(Success)

6. $m.deallocate(p1)$



(With Coalescing)

1. (b)

```

1 p1 = m.allocate(6);
2 p2 = m.allocate(3);
3 p3 = m.allocate(2);
4 m.deallocate(p2);
5 p4 = m.allocate(4);
6 m.deallocate(p1);

```

17 } addresses (indices)
8
12

0.

Initial	
freeStart: 0	
0	15
1	-1
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

1.

(Success)

p1 = m.allocate(6)	
freeStart: 7	
0	7
1	
2	
3	
4	
5	
6	
7	8
8	-1
9	
10	
11	
12	
13	
14	

p1

2.

(Success)

p2 = m.allocate(3)	
freeStart: 11	
0	7
1	
2	
3	
4	
5	
6	
7	4
8	
9	
10	
11	4
12	-1
13	
14	

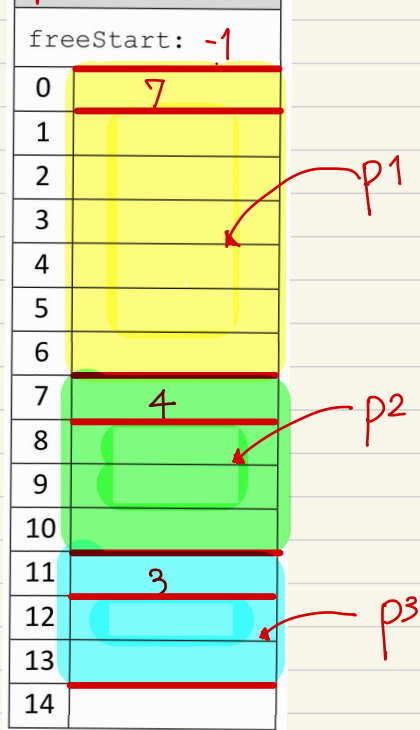
p1

p2

- b. (5 points) Provide a step-by-step demonstration of the allocation/deallocation sequence using the *first-fit algorithm* **with** coalescing. Indicate whether such an allocation/deallocation is successful.

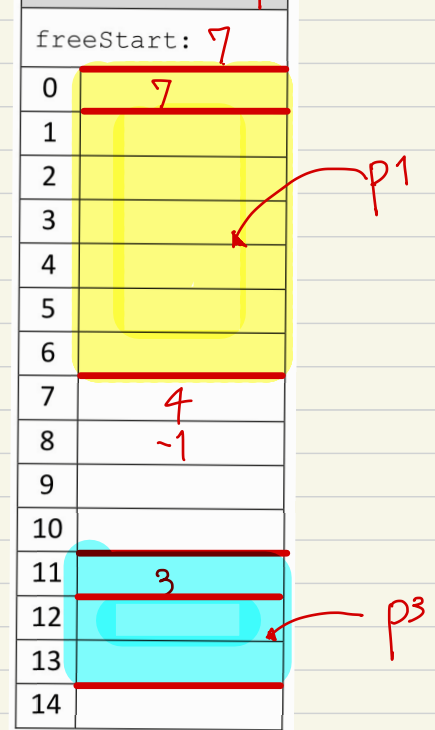
(Success)

3. $p3 = m.allocate(2)$



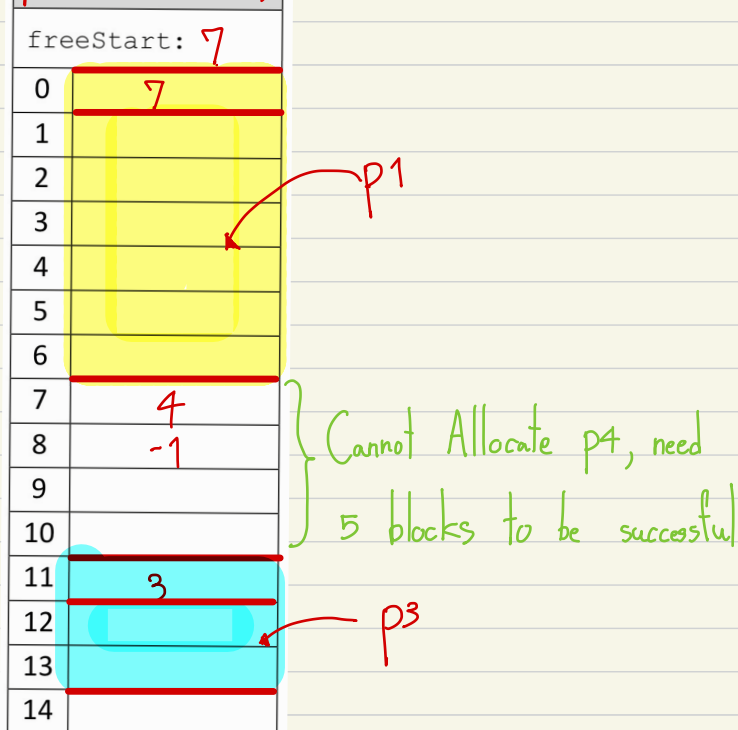
(Success)

4. $m.deallocate(p2)$



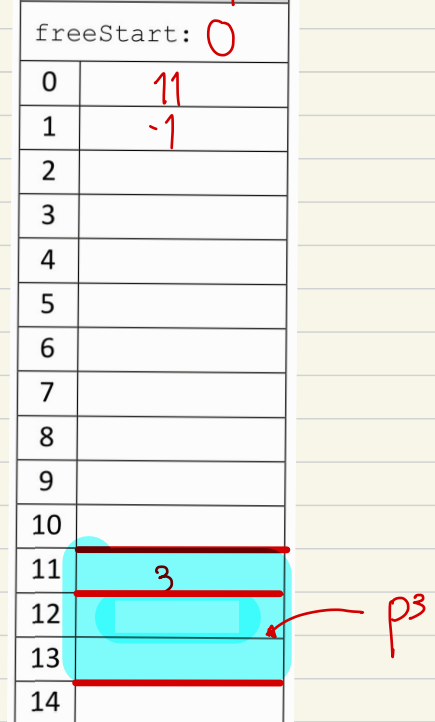
(Fail)

5. $p4 = m.allocate(4)$



(Success)

6. $m.deallocate(p1)$



3.

```
codeshare(cs2203,csx3004).
codeshare(csx3002,itx2001).
takes(james,csx3002).
takes(jane,csx3002).
takes(john,itx2001).
takes(darren,cs2203).
takes(minh,cs2203).
takes(david,csx3004).
takes(pimchanok,csx3004).
takes(ruby,cs3416).
takes(minh,cs3416).
```

takes(minh,cs3416).

a. (5 points) Define a rule `classroster(X,Y)` such that Y is a list of students who take the course X.

b. (5 points) Define a rule `classmate(X,Y)` such that X and Y are classmates if they are taking the same course. Note that courses which are code shared are also considered to be the same. According to the facts given above, james, jane and john are classmates.

CS2203/CSX3004
Darren, Minh, David,
Pimchanok

CSX3002/ITX2001
James, Jane, John

CS3416
Ruby, Minh

```
classroster(X,Y):-
    codeshare(X,X1),
    findall(Z, takes(Z,X), Y1),
    findall(Z1, takes(Z1,X1), Y2),
    append(Y1,Y2, Y).
```

%Simpler classroster
% classroster(X,Y) Y- findall(Z, takes(Z,X), Y).

Course List of student
X
 $\delta = \{X \rightarrow \text{csx3002}\}$

$\text{takes}(Z, \text{csx3002})$
 $\delta = \{Z \rightarrow \text{james}\}$
 $\delta = \{Z \rightarrow \text{jane}\}$

```
%Helper Predicate
samecourse(X,X).
samecourse(X,Y) :- codeshare(X,Y).
samecourse(X,Y) :- codeshare(Y,X).
```

```
classmate(student 1X,student 2Y):-
    takes(X,Z1), course X takes
    takes(Y,Z2), course Y takes
    samecourse(Z1,Z2),
    X \= Y.
```

4 a)

LYNN'S ALGORITHM

4. (10 points) **Tower of Hanoi** is a puzzle game consisting of three pegs and disks in various sizes. The goal of the game is to move all disks from one peg to a different peg while obeying the following rules.

1. Only one disk may be moved at a time.
2. Each move consists of taking the top disk from one of the pegs and placing it on top of another peg or on an empty peg.
3. No disk may be placed on top of a disk that is smaller than it.

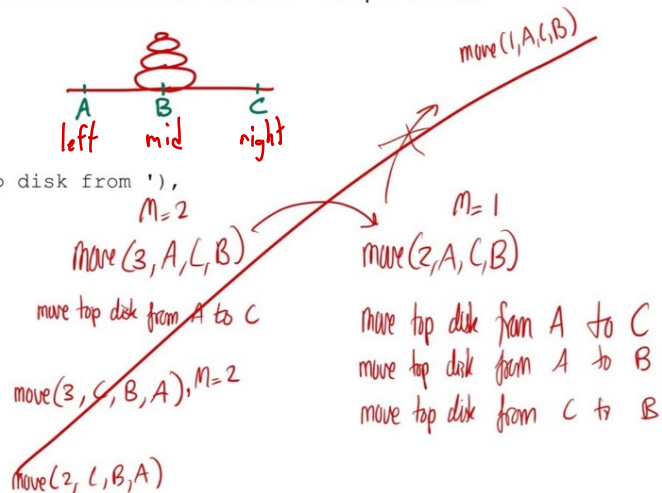
More information can be found at https://en.wikipedia.org/wiki/Tower_of_Hanoi.

Given the following Prolog facts and rules that solve the Towers of Hanoi puzzle below.

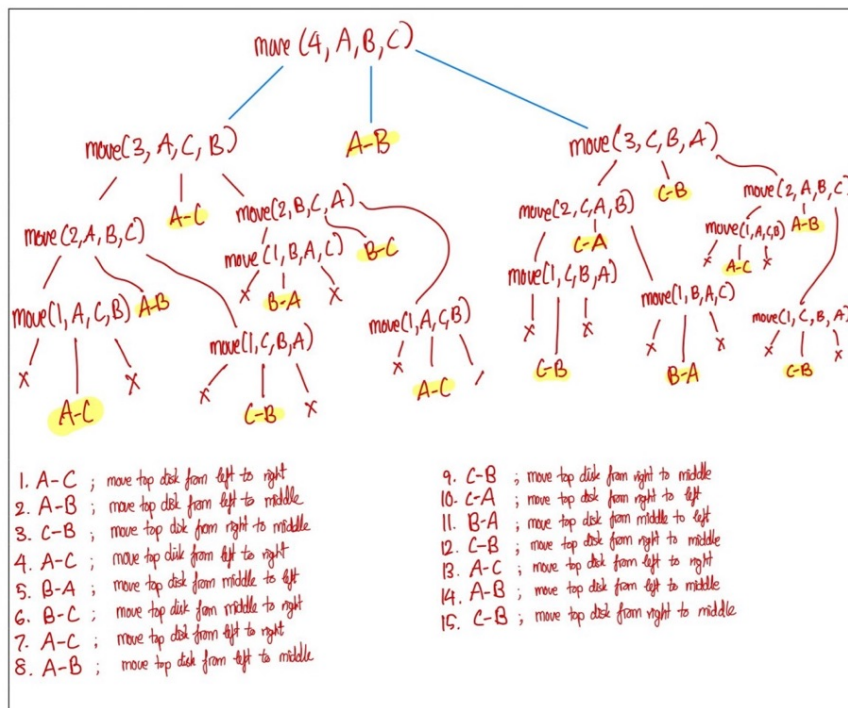
```
peg(left).
peg(middle).
peg(right).
```

```
move(1,A,B,_):-
    write('Move the top disk from '),
    write(A),
    write(' to '),
    write(B),
    nl.
```

```
move(N,A,B,C):-
    N>1,
    M is N-1,
    move(M,A,C,B),
    move(1,A,B,_),
    move(M,C,B,A).
```



a. (5 points) Draw the proof tree for the query `move(4, left, middle, right)`.



Question 4 continues on the next page.

4 b)

```
Move the top disk from left to left
Move the top disk from left to middle
Move the top disk from left to middle
Move the top disk from left to left
Move the top disk from middle to left
Move the top disk from middle to left
Move the top disk from left to left
Move the top disk from left to middle
Move the top disk from left to middle
Move the top disk from left to left
Move the top disk from middle to left
Move the top disk from left to middle
Move the top disk from left to left
Move the top disk from left to middle
Move the top disk from left to middle
true
false
```

b. (5 points) Try the query `move(4, left, right, left)`. Explore the output and identify the problem with the given code. Modify the rules in order to fix such a problem. Show the result after fixing the code.

The problem is that the query `move(4, left, right, left)` does not make sense, since it is impossible to move 4 plates from left to right with the help of left. The solution to this is to add more rules in `move(N, A, B, C)` predicate that says that the variables A, B, C are not the same atom. We can also check whether A, B, C are pegs (According to the facts: only left, middle, right are pegs). If the query fail to satisfy these 2 conditions, then it will return false.

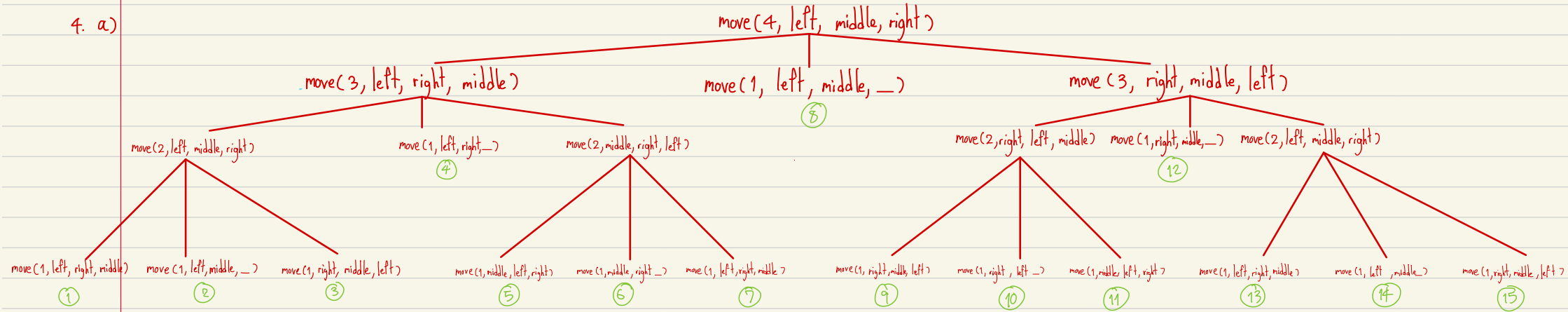
left right middle

```
move(N, A, B, C) :-
    peg(A), peg(B), peg(C),
    A \= B,
    B \= C,
    N > 1,
    M is N - 1,
    move(M, A, C, B),
    move(1, A, B, _),
    move(M, C, B, A).
```

```
move(4, left, middle, left).
false
```

4 plates from left to middle
with help of right

4. a)



4. a)

move(4, left, middle, right)

4 > 1 3 is 4-1 move(3, left, right, middle) move(1, left, middle, —) move(3, right, middle, left)

3 > 1 2 is 3-1 move(2, left, middle, right) move(1, left, right, —) move(2, middle, right, left)

2 > 1 1 is 2-1 move(1, left, right, middle) move(1, left, middle, —) move(1, right, middle, left)

write("Move the top disk from left to right ") n|

write("Move the top disk from left to middle ") n|

write("Move the top disk from right to middle ") n|

write("Move the top disk from left to right ") n|

;
/
/.

write("Move the top disk from to ") n|

3 > 1 2 is 3-1 move(2, right, left, middle) move(1, right, middle, —) move(2, left, middle, right)

;
/
/
/
/
/.

- 2 a) My favorite language is R. It is a multi-paradigm language supporting imperative, object-oriented, and functional. It is strongly typed language, and also dynamically typed. One major part that I like about R is its easy-to-read and minimal syntaxes. This makes R quite easy to pick up for beginners. In addition, R is optimized for matrices, vector calculation and statistics, providing the specific syntaxes need for the computation. For example, matrix multiplication $A \times A$ could be solved by R...

```
A <- rbind(c(1,2), c(3,4))  
print(A %*% A)
```

With just a few line, R could do matrix multiplication, which takes many other languages a lot of line to do.

- b) Currently, R does not support interface, although it supports OOP. The ability to implement interface in R could be useful when dealing with OOP in R.

```
# setInterface("Drawable", methods = list(show))  
  
setClass("Circle", slots = list(radius = "numeric", // implements = Drawable))  
setMethod("show", "Circle", function() {#do show circle})  
  
setClass("Square", slots = list(length = "numeric", // implements = Drawable))  
setMethod("show", "Square", function() {#do show square})
```

